

1.-

1.1.-

Java

```
class MiHebra extends Thread {
    int miId;
    public MiHebra ( int miId ) {
        this.miId = miId;
    }
    public void run ( ) {
        for ( int i = 0 ; i < 1000 ; i++ ) {
            System.out.println ( " Hebra: " + miId ) ;
        }
    }
}
class EjemploCreacionThread {
    public static void main ( String args [ ] ) {
        new MiHebra( 0 ).start();
        new MiHebra( 1 ).start();
    }
}
```

1.2.-

Java

```
class MiRun implements Runnable {
    int miId;
    public MiRun( int miId ) {
        this.miId = miId;
    }
    public void run() {
        for( int i = 0; i < 1000; i++ ) {
            System.out.println( "Hebra: " + miId );
        }
    }
}
class EjemploCreacionRunnable {
    public static void main( String args[] ) {
        new Thread( new MiRun( 0 ) ).start();
        new Thread( new MiRun( 1 ) ).start();
    }
}
```

1.3.- Porque las ids se ven intercaladas.

1.4.- Es una llamada al método, sin la creación de las hebras por lo que ejecutaría el código de forma secuencial.

1.5.- Efectivamente se escribe 1000 veces Hebra: 0 seguido de 1000 Hebra: 1

2.-

2.1.-

Java

```
class MiHebra extends Thread { // (A)
    // (B)
    int miId, num1, num2;
    public MiHebra( int miId, int num1, int num2 ) {
        // (C)
        this.miId = miId;
        this.num1 = num1;
        this.num2 = num2;
    }
}
```

2.2.-

Java

```
// (D)
MiHebra t0 = new MiHebra( 0, 1, 1000000 );
MiHebra t1 = new MiHebra( 0, 1, 1000000 );
t0.start();
t1.start();
```

2.3.- Se observa que el programa principal termina primero, después inician ambas hebras el cálculo y lo terminan juntas.

2.4.- Termina el programa principal sin esperar a las hebras.

2.5.-

Java

```
// (E)
try {
    t0.join();
    t1.join();
} catch( InterruptedException ex ) {
    ex.printStackTrace();
}
```

2.6.- El programa principal espera a que terminen las hebras y después finaliza. No, ya que sean o no Daemon, al hacer join() fuerza a la hebra principal esperar a que dicha hebra termine su proceso.

3.-

3.1.-

Java

```
// (D)
Thread t0 = Thread.startVirtualThread( new MiHebra(0, 1, 1000000));
Thread t1 = Thread.startVirtualThread( new MiHebra(1, 1, 1000000));
t0.run();
t1.run();
```

3.2.- Sin los join, la principal termina enseguida y la JVM se cierra sin esperar a las virtuales.

3.3.- Funcionan siempre como daemon. Por definición lo son y no se puede cambiar.

4.-

4.1.-

Java

```
// =====
class MiHebra extends Thread {
// =====

// Declaracion de variables
int miId;
CuentaIncrementos c;

// -----
// Definicion del constructor, si es necesario
public MiHebra( int miId, CuentaIncrementos c ) {
    this.miId = miId;
    this.c = c;
}

// -----
public void run() {
    System.out.println( "Hebra: " + miId + " Comenzando incrementos" );
    // Bucle de 1000000 incrementos del objeto compartido
    for( int i = 0; i < 1000000; i++ ) {
        c.incrementaContador();
    }
    System.out.println( "Hebra: " + miId + " Terminando incrementos" );
}
}
```

4.2.-

Java

```
// 2. Crear e inicializar el objeto compartido
CuentaIncrementos conta = new CuentaIncrementos();

// 3. Imprimir el valor inicial del contador
System.out.println( "Valor inicial del contador: " + conta.dameContador());

// 4. Crear y arrancar las hebras, utilizando un vector de hebras
MiHebra[] v = new MiHebra[ numHebras ];
for( int i = 0; i < numHebras; i++ ) {
    v[ i ] = new MiHebra( i, conta );
    v[ i ].start();
}

// 5. Esperar a que todas las hebras finalicen
try {
    for( int i = 0; i < numHebras; i++ ) {
        v[ i ].join();
    }
} catch( InterruptedException ex ) {
    System.out.println( "Interrupcion en la espera" );
}

// 6. Imprimir el valor final del contador
System.out.println( "Valor final del contador: " + conta.dameContador() );
```

4.3.-

Sí hay concurrencia, las hebras empiezan y terminan intercaladas en la terminal, lo que demuestra que las hebras se están ejecutando a la vez.

4.4.-

Debería salir 4000000, ya que cada hebra suma 1000000, pero sale entre 3000000 y 1000000.

4.5.-

El objeto no es atómico por lo que hay problemas de visibilidad y un final de carrera ya que las hebras modifican el objeto a la vez, sin tener posibilidad de ver si ha sido cambiado por otra hebra.

5.-

5.1.-

La interfaz no es interactiva, mientras se calcula el número primo no se puede actualizar el número de veces clickadas. Esto ocurre porque la interfaz no funciona de forma concurrente al cálculo del número primo.

5.2.-

Java

```
/* ===== INICIO CODIGO A MODIFICAR EN EJERCICIO 5.2 ===== */
    btnComienzaCalculo = new JButton( "Comienza calculo" );
    btnComienzaCalculo.addActionListener( new ActionListener() {
        public void actionPerformed((ActionEvent e) ) {
            if( txfNumero.getText().trim().length() == 0 ) {
                txfMensajes.setText( "Debes escribir un numero." );
            } else {
                try {
                    // Validacion del numero
                    long numero = Long.parseLong( txfNumero.getText().trim() );
                    // Calculo e impresion en el terminal
                    HebraPrimo hebra = new HebraPrimo( numero );
                    hebra.start();
                } catch( NumberFormatException ex ) {
                    txfMensajes.setText( "No es un numero correcto." );
                }
            }
        }
    });
    tempPanel.add( btnComienzaCalculo );
/* ===== FIN CODIGO A MODIFICAR EN EJERCICIO 5.2 ===== */
...
class HebraPrimo extends Thread {
    long numero;

    public HebraPrimo( long numero ) {
        this.numero = numero;
    }

    public void run() {
        System.out.println( "Examinando numero : " + numero );
        boolean primo = GUIPrimoSencillo.esPrimo( numero );
        if( primo ) {
            System.out.println( "El numero " + numero + " SI es primo." );
        } else {
            System.out.println( "El numero " + numero + " NO es primo." );
        }
    }
}
```

5.3.- Ahora funciona correctamente y como es esperado.

5.4.-

Si son Daemon, al cerrar la ventana la JVM termina inmediatamente y las hebras de cálculo en curso se interrumpen. Por lo tanto es conveniente.